

Space software

Kristoffer Nyborg Gregertsen
TTK4147, NTNU, 2024-10-21

Content

- About SINTEF
- Dependability and safety for space and avionics
 - Standards and methodologies
 - Ada 2012, SPARK 2014 and the Ravenscar profile
 - MISRA C/C++ and Frama-C
- Break?
- I3DS, EROSS and WISER space robotics projects

About me

- SINTEF Digital – Department of Mathematics and Cybernetics
- Research manager for reliable automation group
- PhD in cybernetics from NTNU on real-time system support in Ada with execution time for interrupts
- Now researching on high-integrity software for space applications





SINTEF

ONE OF EUROPE'S LARGEST
INDEPENDENT RESEARCH ORGANISATIONS

Technology for a better society

Vision: Technology for a better society





SINTEF

The UN's Sustainability Goals guides SINTEF

- we must evolve through areas where sustainability is central





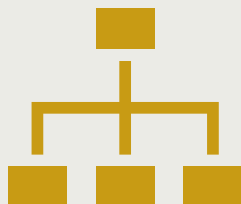
SINTEF

SINTEF Digital – et ledende flerfaglig institutt



475

ANSATTE



6

AVDELINGER



STERKE GENERISKE
FAGMILJØER INNEN
DIGITALE TEKNOLOGIFAG
OG SAMFUNNSFAG



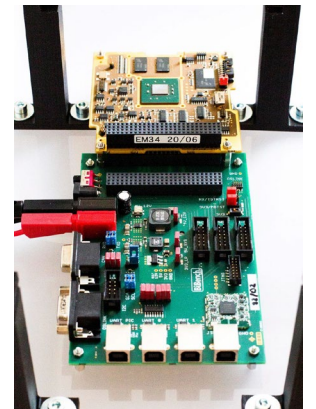
TUNG DOMENE-
KOMPETANSE INNEN
HELSE



SINTEF

Reliable Automation

- Group of 9 researchers in Trondheim
 - Mainly with education in engineering cybernetics
- High-integrity cyber-physical systems (CPS)
 - System integration and safety-critical software
 - Real-time communication, TSN and 5G
 - E.g.: Space robotics and in-orbit servicing with EU/ESA
- Industrial automation
 - Prototyping in robotics and automation
 - IT/OT-integration, sensors, real-time data and AI
 - E.g.: Electrified line fishing system for Mustad Autoline

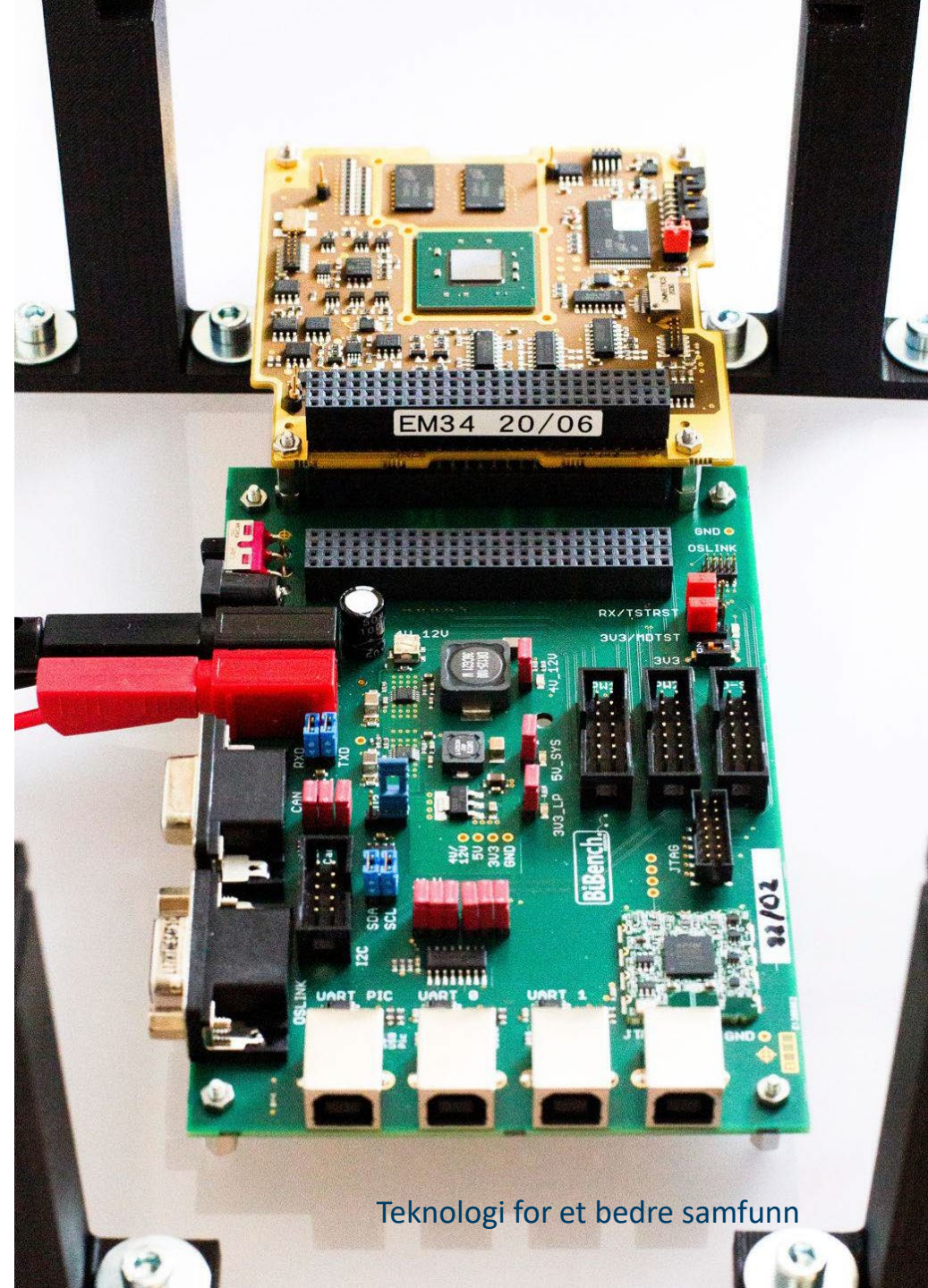




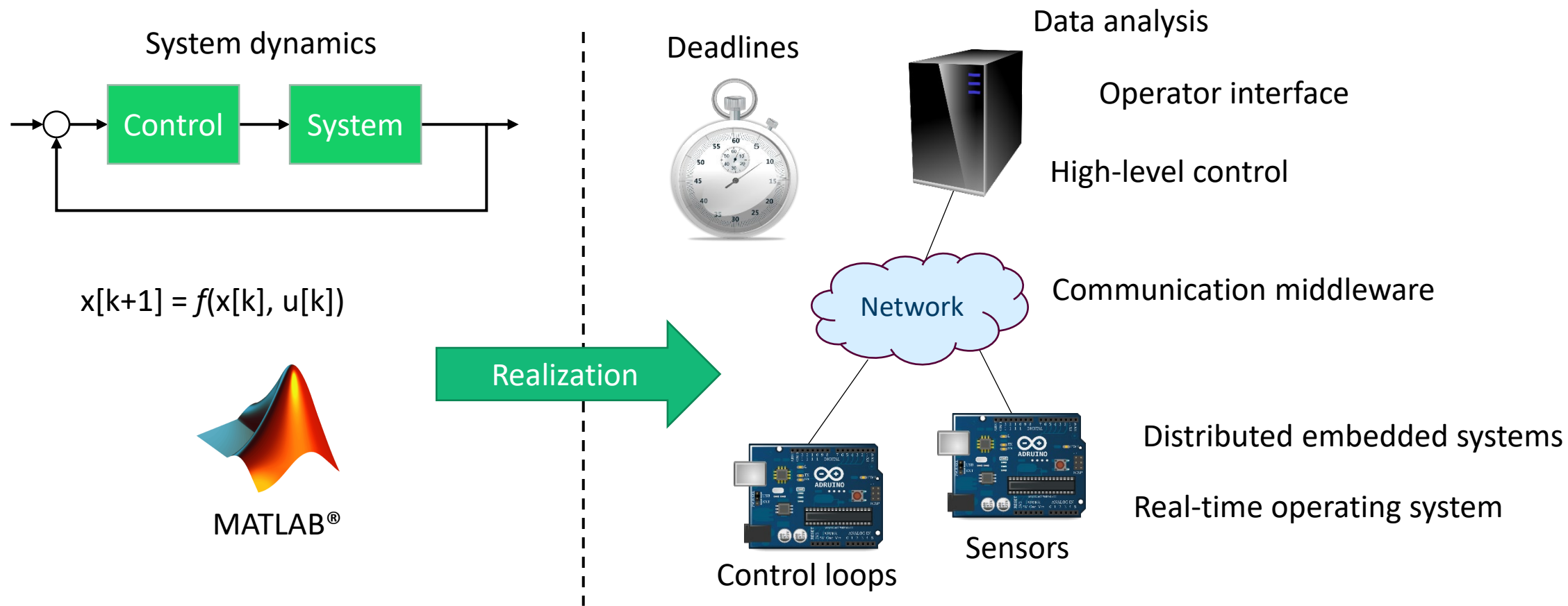
SINTEF

High-integrity CPS

- Strategic research area for the RA group
- We conduct excellent research on high-integrity cyber-physical systems (HI-CPS):
 - Model-based systems engineering (MBSE)
 - Methods for formal verification and testing
 - Requirements, architecture, and verification
 - Dependability, reliability, safety, and real-time
 - Reliable real-time communication
 - Mixed-criticality system integration
 - Real-time operating systems and tools



From model to implementation





SINTEF

Real-time systems

- Correctness of result depends on **time**
 - Systems that interact with physical environment
 - Explicit deadlines – **predictable** execution
- Different classes of criticality:
 - **Soft** real-time systems – some deadline misses tolerable
 - **Firm** real-time systems – result has no value after deadline
 - **Hard** real-time systems – system **failure** to miss deadline



Dependability and safety

- Real-time systems usually also safety-critical:
 - Control physical processes, potentially harmful!
 - **Fault → Error → Failure**
 - Formal verification
 - Error handling
 - Multiple version redundancy...
- Industrial systems need high availability
- Robotics systems may need high reliability



Standards for space applications



- European Cooperation for Space Standardization (ECSS)
 - Standards used by ESA and European national space agencies
 - Open access to standards at <http://ecss.nl> (needs user registration)
- Some central documents for safety-critical software
 - ECSS-S-ST-00-01C – Glossary of terms
 - ECSS-Q-ST-30C Rev.1 – Dependability
 - ECSS-Q-ST-80C Rev.1 – Software product assurance
 - ECSS-E-ST-40C – Software general requirements
- The documents are coherent, reasonable and quite easy to read!



SINTEF

ECSS-S-ST-00-01C definitions

Dependability: The extent to which the fulfilment of a required function can be justifiably trusted. Its main components are reliability, availability and maintainability. Dependability shall be considered in conjunction with safety.

Availability: Ability of an item to be in a state to perform a required function under given conditions at a given instant of time or over a given time interval, assuming that the required external resources are provided.



SINTEF

ECSS-S-ST-00-01C definitions

Reliability: The ability of an item to perform a required function under given conditions for a given time interval. Generally, reliability performance is quantified using appropriate measures. In some applications these measures include an expression of reliability performance as a probability, which is also called reliability.

Maintainability: Ease of performing maintenance on a product. Maintainability can be expressed as the probability that a maintenance action on a product can be carried out within a defined time interval, using stated procedures and resources.

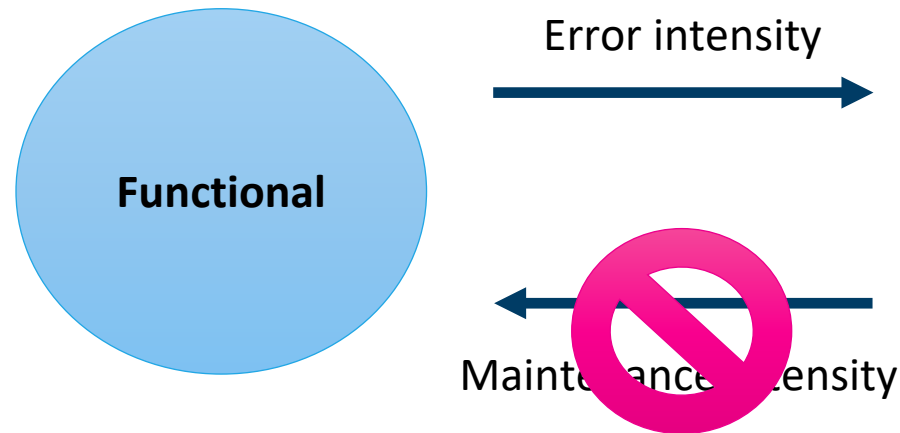


SINTEF

Dependability relationships

Reliability: Time from functional to first failure

Possion intensities → **Markov chain**



Availability: Faction of time in functional state

Maintenance not possible → **Only reliability**



SINTEF

ECSS-S-ST-00-01C definitions

Safety: State where an acceptable level of risk is not exceeded. Here risk relates to fatality, injury or occupational illness; damage to launcher hardware or launch site facilities; damage to an element of an interfacing manned flight system; the main functions of a flight system itself; pollution of the environment, atmosphere or outer space; and damage to public or private property.

Safety critical function: Function that, if lost or degraded, or as a result of incorrect or inadvertent operation, can result in catastrophic or critical consequences.



SINTEF

ECSS-S-ST-00-01C definitions

Certification: Procedure by which a party gives formal assurance that a person or an organization acts, or a product is, in compliance with specified requirements.

Assurance: Planned and systematic activities implemented, and demonstrated as needed, to provide adequate confidence that an entity fulfils its requirements.



SINTEF

ECSS-Q-ST-30C definition of severity

Severity	Level	Dependability consequences
Catastrophic	1	• Failure propagation (for lower-level systems analysis)
Critical	2	• Loss of mission
Major	3	• Major mission degradation
Minor or Negligible	4	• Minor mission degradation or any other effect



SINTEF

ECSS-Q-ST-30C definition of severity

Severity	Level	Safety consequences
Catastrophic	1	• Loss of life, life-threatening or permanently disabling injury or occupational illness. • Loss of system. • Loss of an interfacing manned flight system. • Loss of launch site facilities. • Severe detrimental environmental effects.
Critical	2	• Temporarily disabling but not life-threatening injury, or temporary occupational illness. • Major damage to an interfacing flight system • Major damage to ground facilities • Major damage to public or private property • Major detrimental environmental effects
Major	3	---
Minor or Negligible	4	---



ECSS-Q-ST-30C definition of criticality

Severity	Function criticality	Criteria to assign criticality categories to functions
Catastrophic (Level 1)	I	A function that if not or incorrectly performed, or whose anomalous behaviour can cause one or more feared events resulting in catastrophic consequences .
Critical (Level 2)	II	A function that if not or incorrectly performed, or whose anomalous behaviour can cause one or more feared events resulting in critical consequences .
Major (Level 3)	III	A function that if not or incorrectly performed , or whose anomalous behaviour can cause one or more feared events resulting in major consequences .
Minor or Negligible (Level 4)	IV	A function that if not or incorrectly performed , or whose anomalous behaviour can cause one or more feared events resulting in minor or negligible consequences .



SINTEF

Discuss!

Does safety
considerations apply
to unmanned
missions?

What about
dependability?



SINTEF

ECSS-Q-ST-80C software assurance

- **Goal:** Provide adequate confidence to the customer and to the supplier that the developed or procured/reused software satisfies its requirements throughout the system lifetime.
- Defines requirements for software product assurance to meet this:
 - Organization and responsibility
 - Procurement
 - Tools, methodologies, metrics
 - Training
- Specifies the required plans and reports for different project stages



SINTEF

ECSS-Q-ST-80C software categories

- The standard defines four software criticality categories:
 - **A:** Software involved in category I functions, **AND** no compensating provisions exist. **OR** the software is included in compensating provisions for category I functions.
 - **B:** Software involved in category I functions **AND** at least one of the following compensating provisions is available: a hardware implementation, a software implementation of category A, or an operational procedure. **OR** the software is involved in category II functions **AND** no compensating provisions exist. **OR** the software is included in compensating provisions for category II functions.
 - **C:** Same as above but for category II and III.
 - **D:** Same as above but for category III and IV.



SINTEF

ECSS-E-ST-40C general requirements

- General software requirements for different stages of project:
 - Development, verification and validation
 - Operation and maintenance
- Specifies requirements for different software categories such as
 - Code test coverage (100% for A/B, only by agreement for C/D)
 - Real-time schedulability analysis
 - Formal proof
- Specifies the required plans and reports for different project stages



SINTEF

Civil aviation software standards

- **DO-178C:** Software Considerations in Airborne Systems and Equipment Certification. Defines **Software Level** or **Design Assurance Level (DAL)**, documentation in certification artifacts, Low- and High-Level Requirements, traceability analysis...
- Supplements to standard:
 - **DO-330:** Software Tool Qualification Considerations
 - **DO-331:** Model-Based Development and Verification
 - **DO-332:** Object-Oriented Technology and Related Techniques
 - **DO-333:** Formal Methods



SINTEF

Summary of software standards

- Product, functionalities, life-time, dependability and safety
- Specify the criticalities, severities, and assurance levels
- Specify how the software assurance process is done
- Specify how the software development process is performed
- Plans, audits, certification, training, maintenance...
- **LOTS** of documentation involved!



SINTEF

Programming languages and tools

- Programming languages and tools for safety-critical systems
 - Ada 2012 programming language and SPARK 2014
 - C programming language, MISRA C/C++ and Frama-C
 - Rust programming language, new kid on the block, not further mentioned, but also important
- Compilers, verification tools, run-times, RTOS, certification
- Should avoid prejudice and "holy language wars"
- What is the best tool for the job?



SINTEF

Ada programming language

- Ada is a general-purpose programming language that has been specifically designed for safe and secure programming.
- Originates from late 70s where US DoD ordered a programming language to replace hundreds of languages used for its projects.
- First ISO-standard was Ada 83, current standard is Ada 2022.
- While it is general purpose, Ada is today most used in safety-critical domains such as avionics, space, train control and similar.





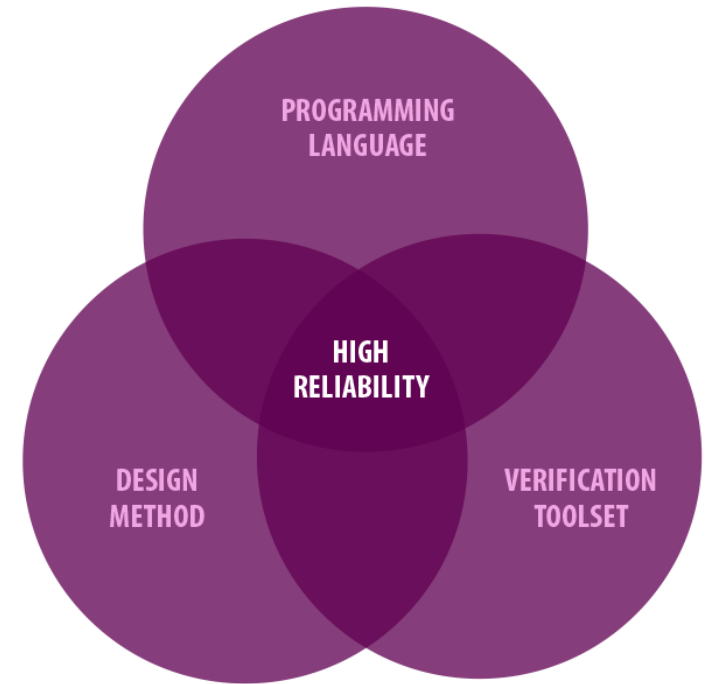
SINTEF

Some features in Ada 2022

- Strong typing
- Rich type system
 - Integer and floating-point ranges
 - Array and record types
 - Access types
- Object-orientation
- Large system with packages
- Tasking support in language
 - Tasks, priorities and scheduling
 - Task rendezvous and protected objects
 - Interrupt handling
- High-precision clocks
- Exception handling
- Programming by contract

SPARK 2014

- Sub-set of Ada 2022 for formal analysis:
 - No general access types, limited support with ownership control
 - No side effects for functions and expressions
 - No exceptions and limited tasking (Ravenscar profile)
- Uses the Ada 2022 aspect feature:
 - Flow-control for variables, including (encapsulated) global variables
 - Contracts for functions and procedures
- Basic usage proves program flow and absence of run-time errors
- Full usage gives formal proof of program properties



Flow-control



"Silver level"

- Analyse flow of variables
 - Specify input and output of subprograms and how they relate
 - Uninitialized data and unused data
- Includes analysis of global variables
 - Common cause of errors in large systems
 - Also global variables that are private in packages
- Volatile variables with effective reads and writes
 - For instance used for device drivers
- Abstract state for packages, refined in private implementation

Contracts and formal proof

- Use pre/post-condition aspects in Ada 2022
 - What must be true when the subprogram is called
 - What must be true after the subprogram has been called
 - Helper pragmas to assist the formal proof
- Also for object-orientated class threes
 - Pre-conditions must be same or weaker than root class
 - Post-condition must be same or stronger than root class
- Prover examines conditions versus source code



"Gold level"

```

package Mileage
  with Abstract_State => (Trip,  -- number of miles so far on this trip (can be reset to 0)
                          Total) -- total mileage of vehicle since last factory-reset
is
  function Trip  return Natural;  -- Has an implicit Global => Trip
  function Total return Natural;  -- Has an implicit Global => Total

  procedure Zero_Trip
    with Global  => (Output => Trip),  -- In the Global and Depends aspects
         Depends => (Trip => null),    -- Trip denotes the state abstraction.
         Post    => Trip = 0;         -- In the Post condition Trip denotes
                                     -- the function.

  procedure Inc
    with Global  => (In_Out => (Trip, Total)),
         Depends => ((Trip, Total) =>+ null),
         Post    => Trip = Trip'Old + 1 and Total = Total'Old + 1;

  -- Trip and Old in the Post conditions denote functions but these
  -- represent the state abstractions in Global and Depends specifications.

end Mileage;

```

Ravenscar profile

- The full concurrent constructs of Ada are non-deterministic and thus unsuited for formal analysis of real-time properties
- The Ravenscar profile is a sub-set of the concurrent constructs:
 - Static set of tasks and communication channels
 - Single entry per protected object with ceiling locking
 - No task rendezvous, requeue or dynamic priorities
 - Supported with SPARK 2014
- Allows rate monotonic analysis of real-time properties
- As side effect, allows very compact run-time environments

AdaCore has run-time
certified for DO-178C and
MP run-time for Level A



SINTEF

C/C++ programming languages

- The C programming language is **inherently unsafe**
 - Very low level of abstraction, for instance access to memory
 - No strong checking of type
 - Very easy, efficient and supported on every target
- Standard C++ is perhaps worse by adding complex functionality
 - Templates, inheritance, polymorphism, move semantics and operator overloading...
- Being a *good* C/C++ programmer takes a lot of training and skill
- MISRA C/C++ tries to counter the weaknesses by adding rules

MISRA C

- A set of software development guidelines for C developed by MISRA (Motor Industry Software Reliability Association)
- To be compliant to the MISRA C Guidelines, all *mandatory* rules shall be met and all *required* rules and directives shall either be met or subject to a formal deviation
- There exists tools to check source code for compliance
- Not a free (gratis) standard, but only cost £15 per PDF copy.



SINTEF

Some rules in MISRA C

Rule 1 (required): All code shall conform to ISO 9899 standard C, with no extensions permitted.

Rule 20 (required): All object and function identifiers shall be declared before use.

Rule 49 (advisory): Tests of a value against zero should be made explicit, unless the operand is effectively Boolean.

Rule 104 (required): Non-constant pointers to functions shall not be used.

Frama-C

- Has similar functionality as SPARK 2014
- Use the same theoretical basis for proof engines
- Notations provided as comments with special format
 - Assertions
 - Contracts with pre- and post-conditions
- Also allows usage of pointers, was not allowed in SPARK 2014
- Primary difference between the two is the underlying language



SINTEF

Comparison of annotation

```
/*@  
requires \valid(x);  
requires *x < 2147483647;  
assigns *x;  
ensures *x == \old(*x) + 1;  
*/  
void increment_guarded (int *x)  
{  
    (*x)++;  
}
```

```
procedure Increment_Guarded  
  (X : in out Integer)  
    with  
      Pre  => X < Integer'Last,  
      Post => X = X'Old + 1  
  
  is  
begin  
    X := X + 1;  
end Increment_Guarded;
```



SINTEF

RTOS for C/C++

- C does not specify multitasking, although POSIX is common
- Many different RTOS for embedded systems exist for C/C++
 - RTEMS, open-source, very popular in space applications
 - FreeRTOS, open-source, very popular for embedded systems
 - SafeRTOS, variant of FreeRTOS for safety-critical
 - VxWorks, proprietary
 - QNX, proprietary
 - PikeOS, proprietary
- A few of these are available in certified versions



SINTEF

Summary on languages and tools

- Ada and C/C++ are commonly used for safety-critical programming
 - Only Ada has been designed for safety-critical systems
 - C/C++ relies on coding guidelines such as MISRA (or JSF)
- Static analysis – sound tool finds all defects, unsound only helps
 - Both SPARK and Frama-C are sound tools that provide functional proof
 - Find absence of run-time errors, and prove that code works as specified
- Pay attention to a fully certified eco-system with compiler, verification tools and multi-tasking run-time



SINTEF

References

<http://docs.adacore.com/spark2014-docs/html/lrm/index.html>

<https://www.embedded.com/electronics-blogs/beginner-s-corner/4023981/Introduction-to-MISRA-C>

<http://www.cubesatlab.org:430/PUBLIC/SPARK-Frama-C-Day-2017.pdf>

<https://www.adacore.com/uploads/technical-papers/2016-10-SPARK-MisraC-FramaC.pdf>

<http://www.stroustrup.com/JSF-AV-rules.pdf>

Additional reading:

https://www.nasa.gov/sites/default/files/atoms/files/improving_mission_success_of_cubesats_-_tor-2017-01689.pdf

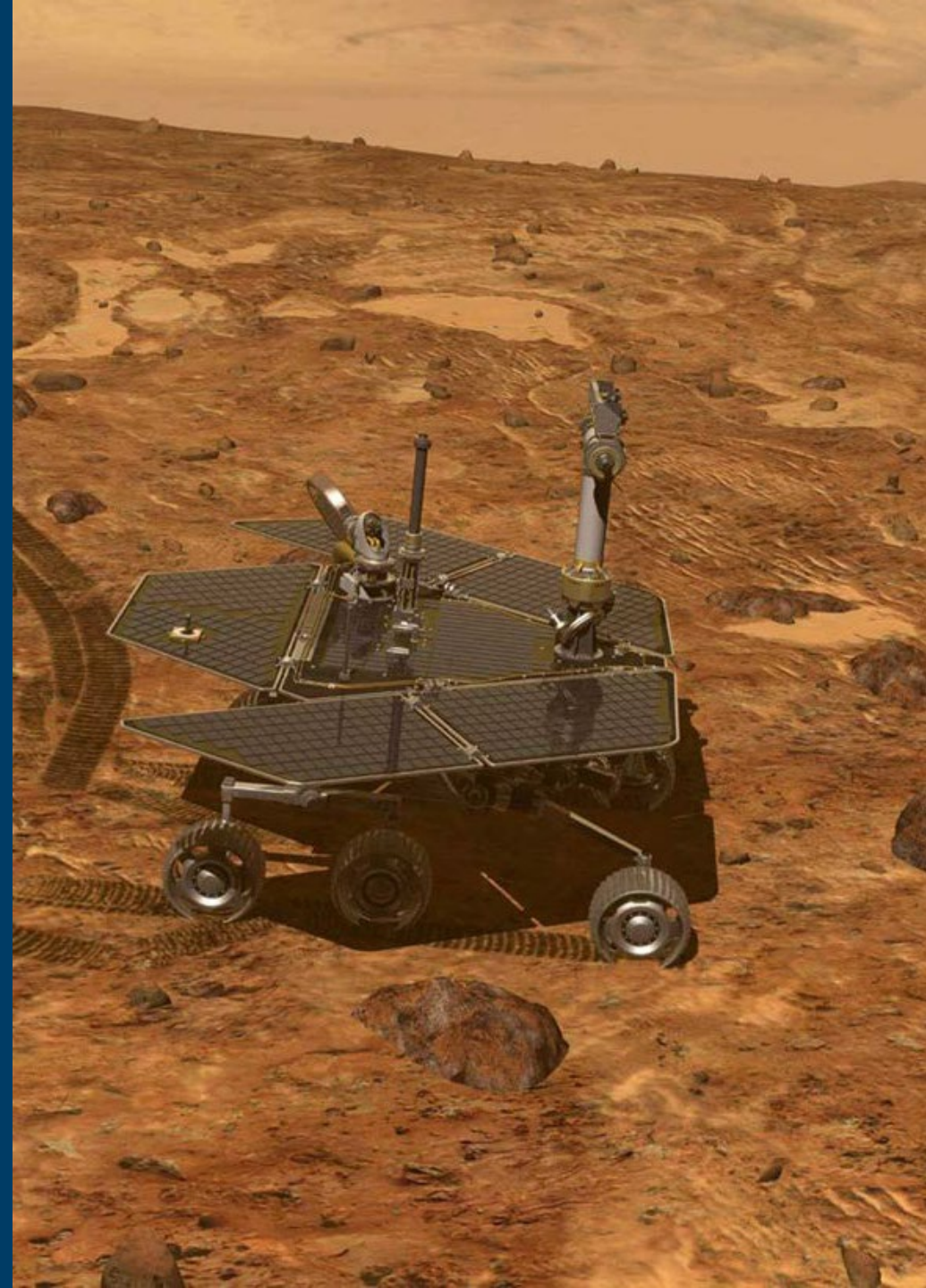


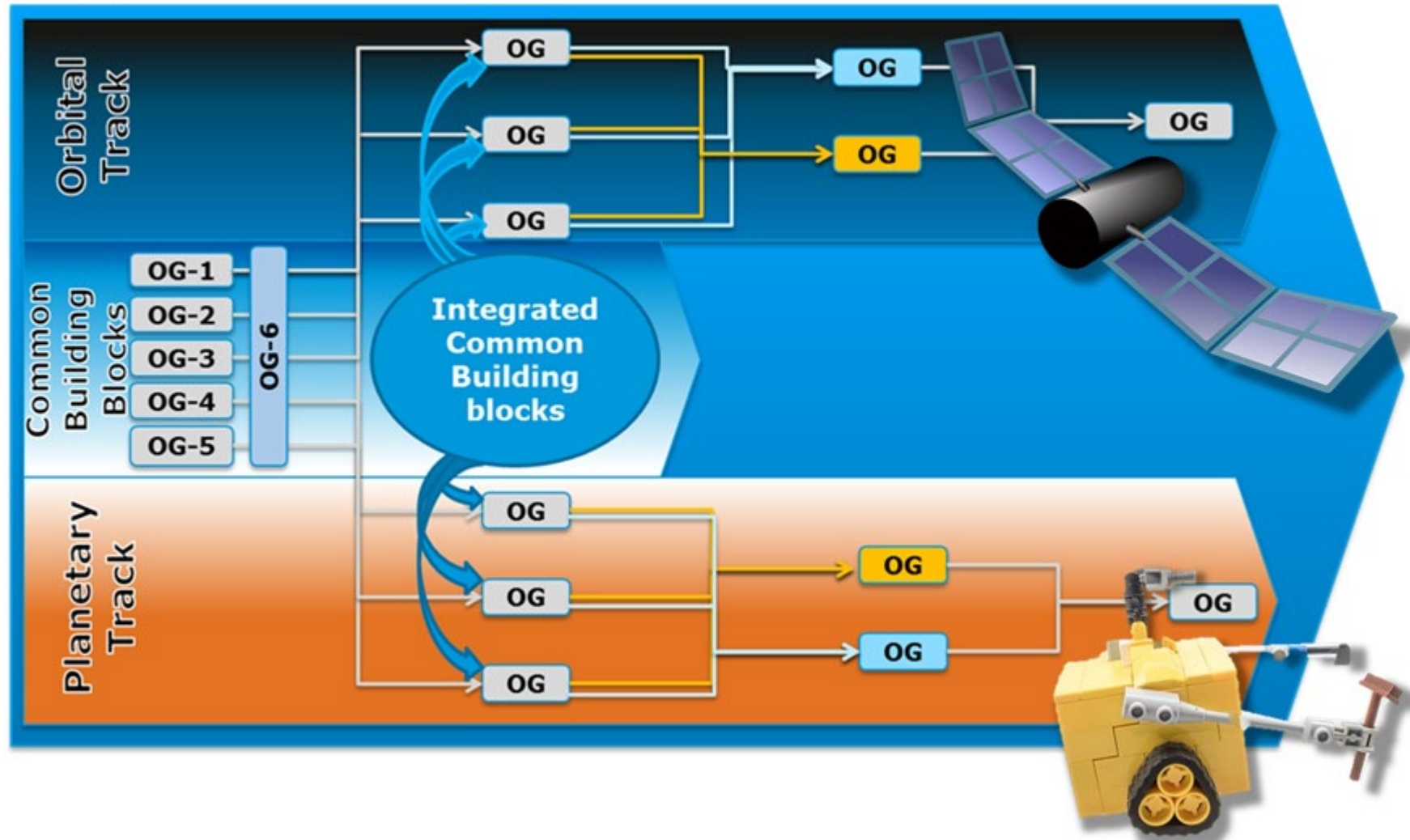
SINTEF

Teknologi for et bedre samfunn

Space robotics

- Space Robotics Cluster
 - Future robotics platform for ESA
 - Led by the PERASPERA project
 - Funded by the European Commission
 - Parallel Operational Grants (OG's)
- I3DS, EROSS, EROSS+, EROSS IOD
 - Thales Alenia Space is coordinator
 - SINTEF leads software development, integration and interface definition
 - **Launch space demonstrator**





I3DS project motivation and goals

- I3DS: Integrated 3D Sensors
- Develop and demonstrate a modular sensor suite for space robotics
 - Tight cooperation with OG's for middleware, autonomy and sensor fusion
 - Standardized sensor interfaces with ASN.1 messages for system integration
 - Sensors, Instrument Control Unit (ICU) and software at **TRL5**
- Motivation: Reduce development time of space missions
 - Abstract away device specific details with standard interface for sensor class
 - Allows to use latest sensors available without changing other software
 - Reuse sensor interfaces, processing components and ICU hardware



Mission-specific
sensor suite with
standard components

- European Robotic Orbital Support Services – In-Orbit Demonstrator
 - Goal: extending satellite lifetimes while limiting space debris to enable better management of the current and future space assets
 - Future space systems will be able to undergo maintenance and upgrades even in orbit
- EROSS IOD will validate the technologies needed for robotic in-space servicing operations, culminating in a European pioneering mission by end of decade
- Consortium of 17 companies, led by Thales Alenia Space in France, 100% funded by EU through Horizon Europe programme
- SINTEF Digital is designing and developing the standardized interfaces of the Equipment Abstraction Layer (EquAL) as well as a novel Flash LIDAR 3D sensor

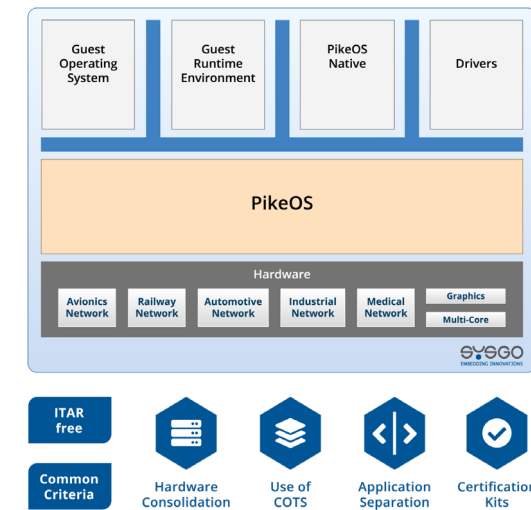
Robotic Control Unit (RCU)



- RCU build on the Xilinx Zynq UltraScale+ MPSoC
 - Mixed-criticality real-time system
 - Quad-core ARM Cortex A53 with PetaLinux / PikeOS
 - Two ARM Cortex R5 real-time processors
 - FPGA for bespoke hardware modules
 - ARM Mail GPU for processing with support for OpenCV
- Runs sensors interfaces and processing algorithms
 - Hardware acceleration of visual navigation
- **The Xilinx Zynq UltraScale+ is a complex platform!**



PikeOS hypervisor / RTOS

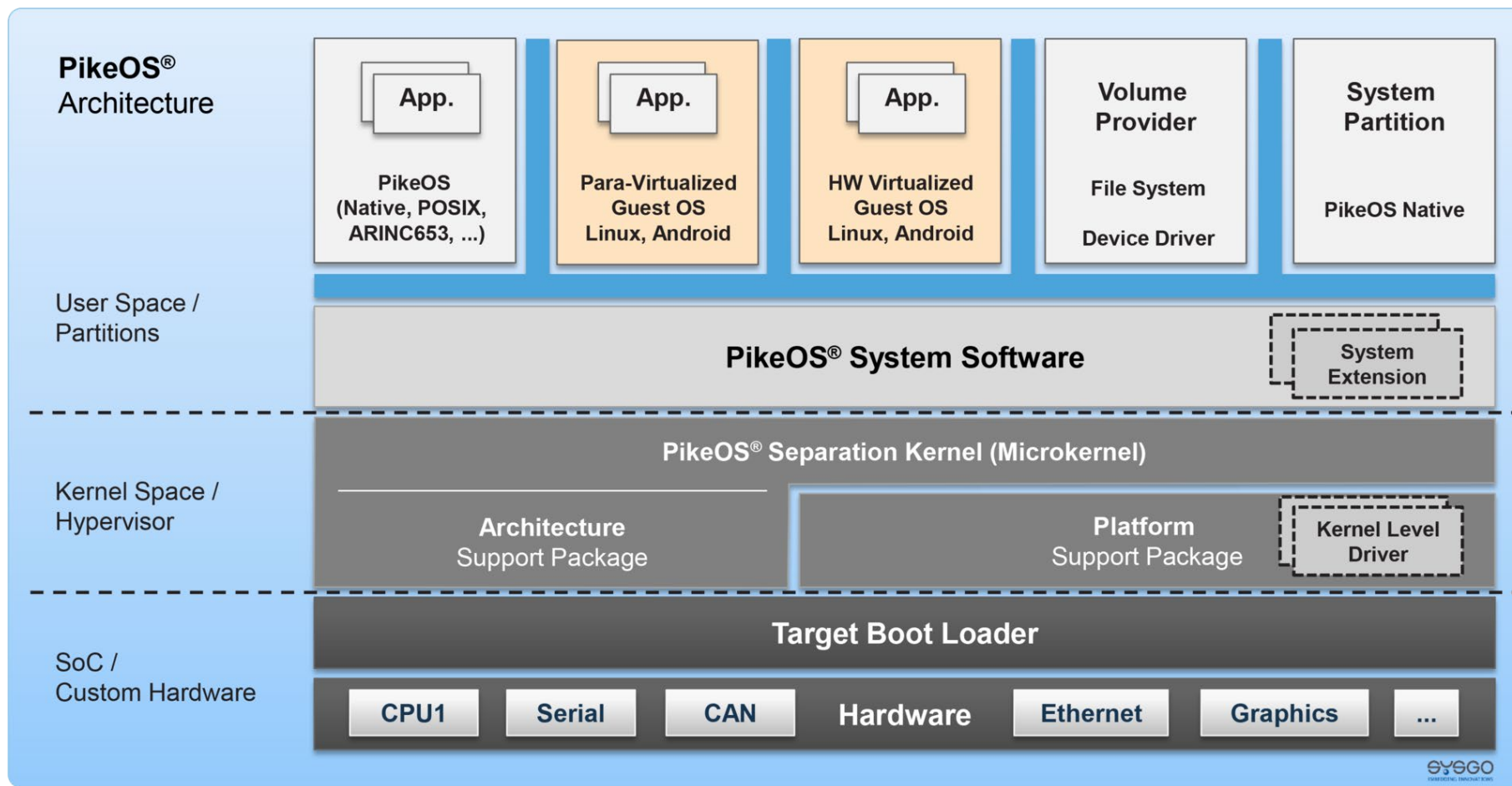


- PikeOS is a hypervisor and RTOS developed by SYSGO
- Proprietary and €€€expensive
- Based on the L4 family of microkernel systems
- Very small kernel with message passing to services
- Certified for SIL-4 / DAL-A separation of partitions
- Memory / resource separation
- Temporal separation
 - Real-time scheduling
 - Time-slicing on processor
 - Bind threads to processors



SINTEF

PikeOS architecture and personalities



Project Structure

type filter text

- ▼ pikeos-native-devel
 - [-] Compilation Parameters
 - ▶ PikeOS
 - ▶ Default
 - ▶ Development Tools
 - ▶ TAS Platform RCU
 - ▼ Robot CTL
 - ▶ Robot CTL Partition [modified VMIT]
 - ▼ Robot CTL Process
 - [-] Advanced Process Settings
 - ▼ Robot SKE
 - ▶ Robot SKE Partition [modified VMIT]
 - ▶ Robot SKE Process
 - ▼ Robot IP
 - ▶ Robot IP Partition [modified VMIT]
 - ▶ Robot IP Process
 - ▼ Robot NAV
 - ▶ Robot NAV Partition [modified VMIT]
 - ▶ Robot NAV Process
 - ▼ RSM ERGO
 - ▶ RSM ERGO Partition [modified VMIT]

Add...

New Group

Rename

Remove

Up

Down

Dependencies

Assigned

Description

This is a PikeOS integration project.

In this project you should:

- select the board your image will run on.
- define partitions and applications.
- define communication means (shm, ports, files etc) and drivers.

Romimage parameters

☐ Romimage checksum ☐ false

Installation

Here you can define the name of the generated image and where it will be installed during **make install**

If **Image name** is empty, a name containing this project name, board and boot strategy is automatically generated.

To install as /tftpboot/uiimage you should:

- set **Image Name** to uiimage
- set **Installation Directory** to FILESYSTEM/tftpboot

Image Name pikeos_scu
 Installation Directory CUSTOM_POOL/images

Details

Product: PikeOS

Version: 5.1

Profile: integration

Board: TAS Platform RCU

Target: arm_v8hf

Arch: arm

Proc: v8hf

Paths

PikeOS: /opt/pikeos-5.1/

PikeOS Pool: /opt/pikeos-5.1/target/arm/v8hf/

Custom Pool: /home/kristoffern/Workspace/PikeOS/pool/

Browse...

Workspace...

Channels

pikeos

service

robot_ip_partition

robot_nav_partition

rsm_ergo_partition

robot_ctl_partition

can_out

robot_state

robot_ske_partition

rdv_ip_partition

i3ds_partition

can_out

mon_master

ctrl

mon_ssw

ctrl

hmev

pikeos

service

robot_ip_partition

robot_nav_partition

S robot_state

rsm_ergo_partition

robot_ctl_partition

Q can_in

robot_ske_partition

S robot_state

rdv_ip_partition

i3ds_partition

Q can_in

mon_master

ctrl

mon_ssw

ctrl

Channel Table

Type	Port Type	Part. ID >	Port >	Provider	Part. ID <	Port <
Partition	VM_PORT_SAMPLING	4	robot_state	<N/A>	5	robot_state
Partition	VM_PORT_SAMPLING	4	robot_state	<N/A>	3	robot_state
Partition	VM_PORT_QUEUEING	4	can_out	<N/A>	10	can_in
Partition	VM_PORT_QUEUEING	10	can_out	<N/A>	4	can_in

CreateDelete

Attributes

Name	Value
TEXT Name	can_in
123 Max Message Count	16
123 Max Message Size	14
BIT Direction	VM_PORT_DESTINATION

Overview

VMIT

Scheduling

Channels

SharedMemory

Romimage

Source



SINTEF

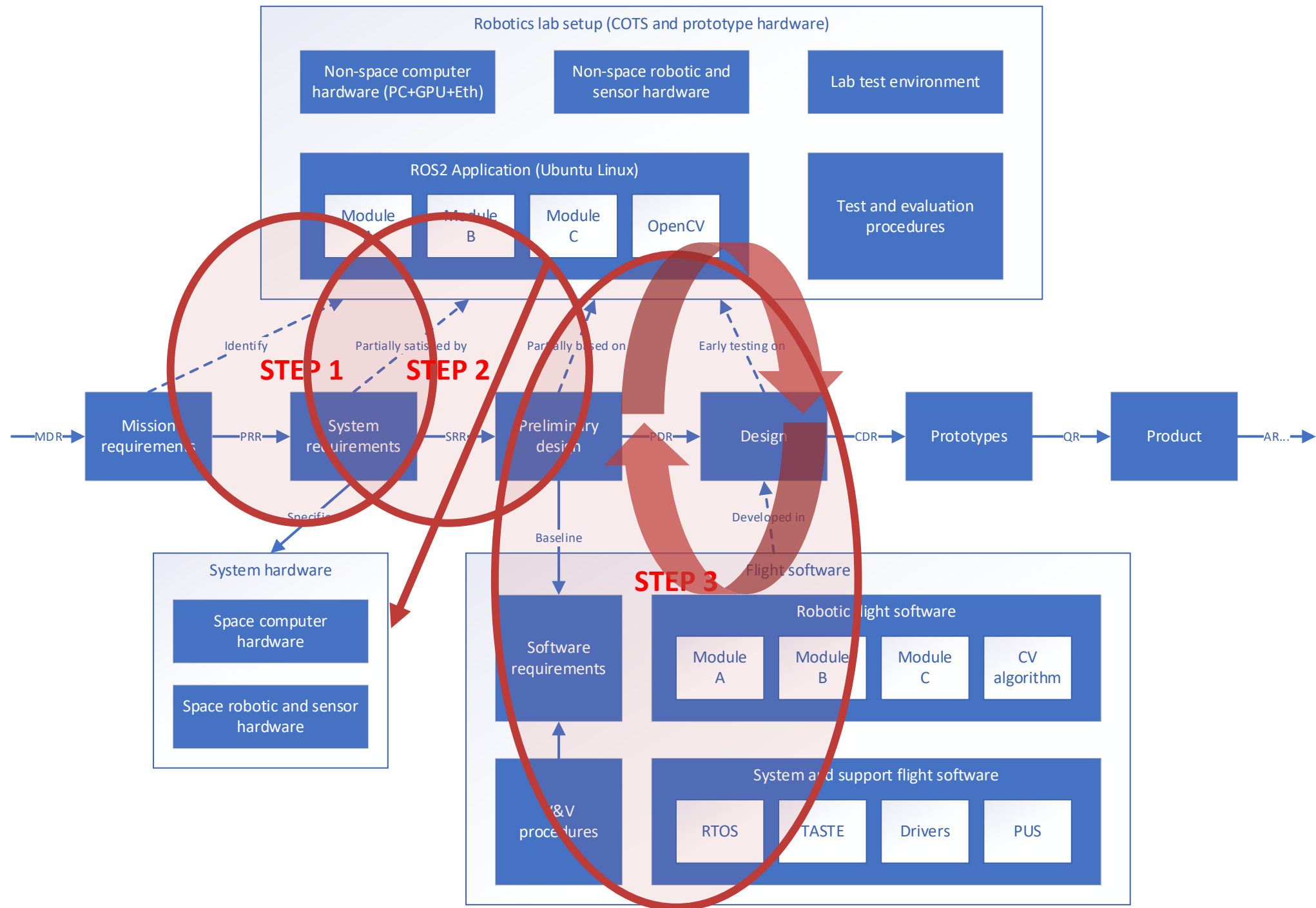
Ada and SPARK 2014 in EROSS

- Ember – Embedded robotics library
 - Utilities needed for robotics and control systems developed through EROSS
 - Linear algebra, complex numbers and quaternions, kinematics
- EquAL – Equipment Abstraction Layer
 - Architecture and utilities for abstraction of different robotic equipment
 - Utilities for communication and data serialization
- Developed with qualification for Level B in mind
 - Formal verification with SPARK 2014 – also need unit-testing for numerical algorithms
 - Flow control and dependencies for data processing
 - SPARK 2014 allows us to develop code with confidence!



WISER project

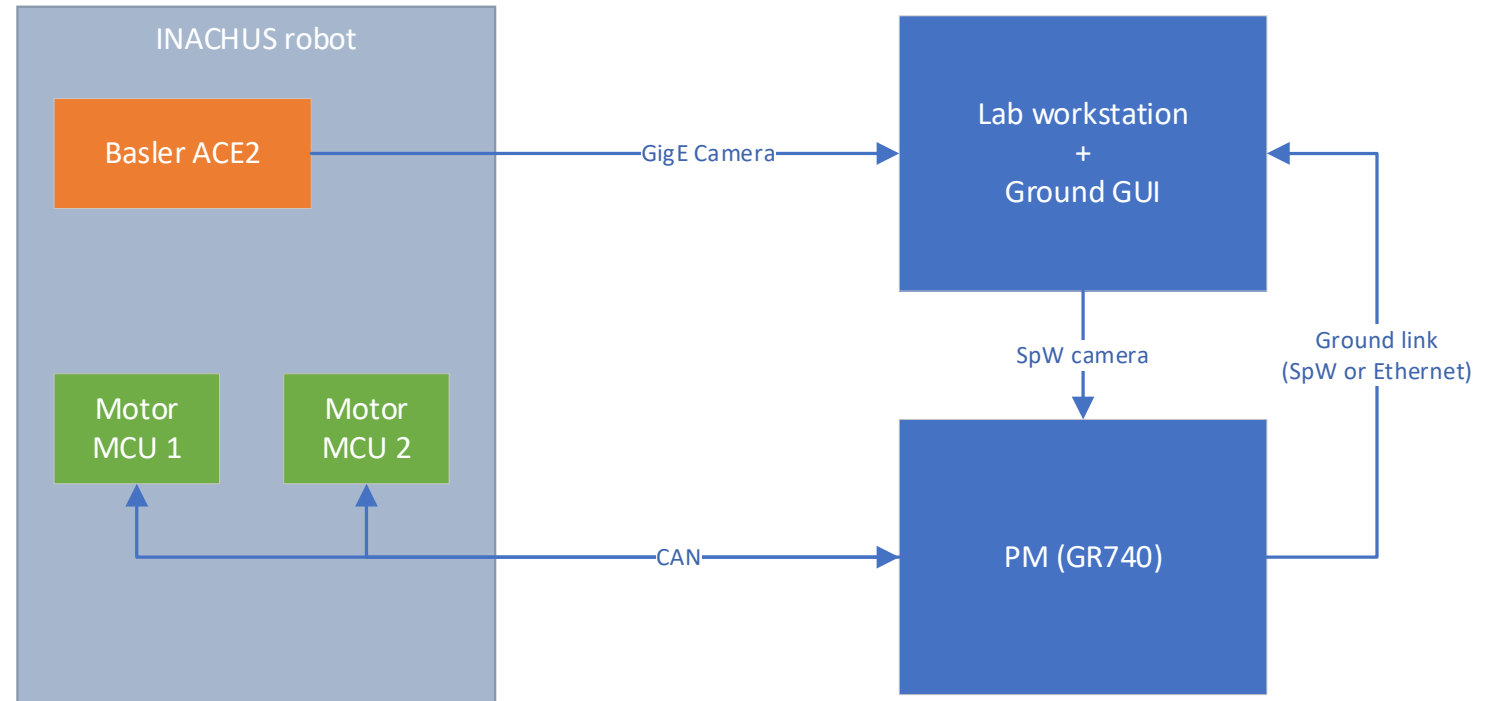
- Newly started ESA-project
 - Workflow for Industrialization of Software produced in Experimental Robotics
 - SINTEF is only project partner
 - Duration 2024 to 2026
- Research goal: How to get ROS2 robotic prototypes from the lab and into space faster and more cost efficient?
- Develop the WISER workflow:
 1. Develop and demonstrate the application in ROS2 in a lab PC environment.
 2. Adapt the application to space computing- and network hardware support.
 3. Port the software according to ECSS assurance procedures.
- Tailoring of the ECSS standards?
- Integration between ROS2 and TASTE



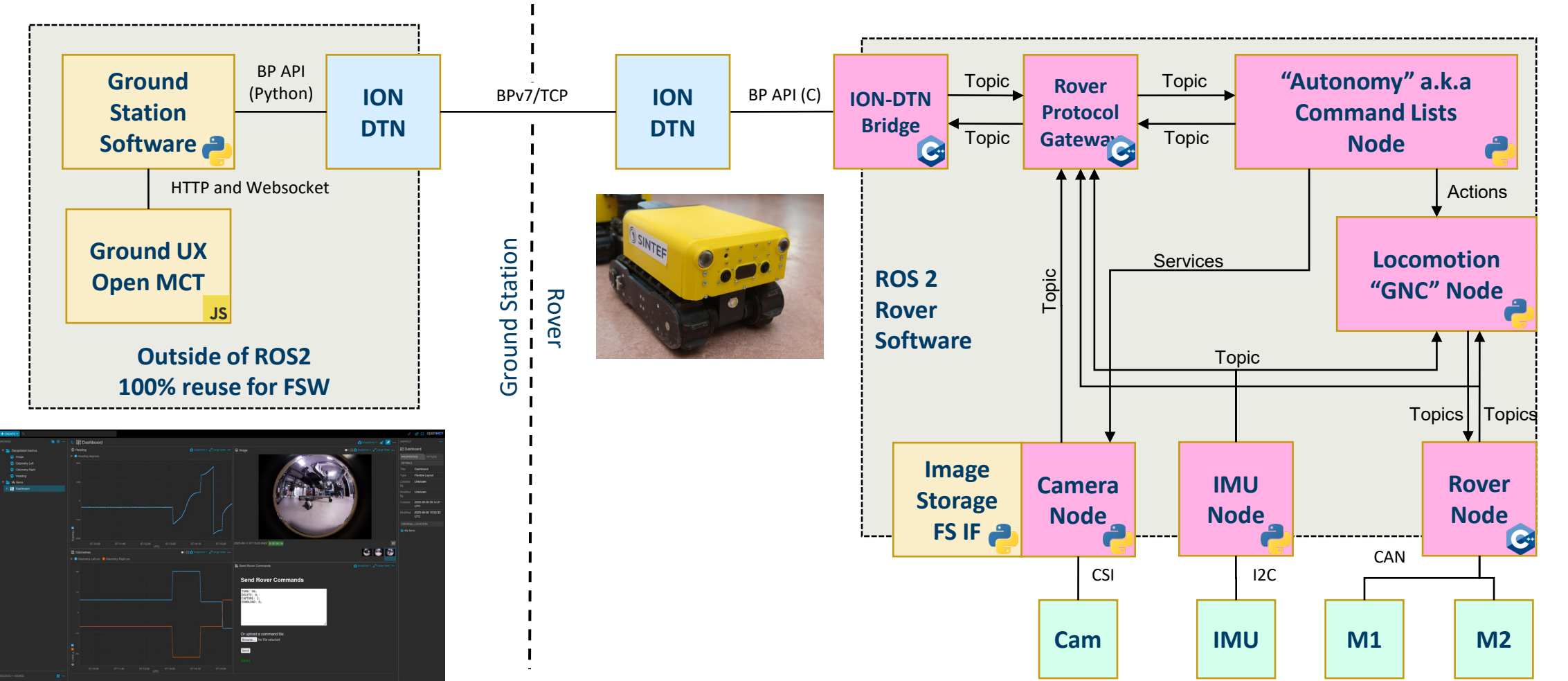


SINTEF

Sprint 1: Rover locomotion and teleoperation



Implemented ROS2 Architecture



rover-command/command_exe

My Items

Dashboard

←

→

↺

🏠

Not secure

http://10.222.2.26:8000/#/browse/mine/4df117b8-d8f1-438d-800a-30d30714d586?tc.mode=local&tc.startDelta=300000&tc.endDelta=0&tc.timeSystem=utc&view=flexible-layout

📖

Verktøy med supers...

★

Bookmarks

🌐

UNO Forside – SINI...

🌐

Uno

🌐

Styret og IU interne...

🌐

Home Assistant

🌐

Luftkvalitet i Norge

🌐

Hus - Grafana

🌐

Home - Grafana

🌐

mode

🌐

Inspira Assessment

🌐

Work items - Boards

🌐

CURSOR Home - C...

🌐

KarlGarmin

🌐

sync

+ CREATE

🔍

BROWSE

Decapitated Inachus

Image

Odometry Left

Odometry Right

Heading

My Items

Dashboard

Heading

Image

Odometries

Send Rover Commands

Dashboard

Snapshot

Large View

Heading

Heading degrees



Heading

08:50:00 08:53:20 08:56:40 09:00:00 09:03:20 UTC

Odometries

Odometry Left cm Odometry Right cm



Y Axis 1

08:50:00 08:51:40 08:53:20 08:55:00 08:56:40 08:58:20 09:00:00 UTC

Image



2025-09-23 08:56:59.409Z 00:00:39

Send Rover Commands

Send Rover Commands

MOVE: 20;
TURN: -90;
MOVE: 20;
TURN: 90;
MOVE: 40;
TURN: 90;
MOVE: 20;
TURN: -90
CAPTURE: 2;
DOWNLOAD: 0;

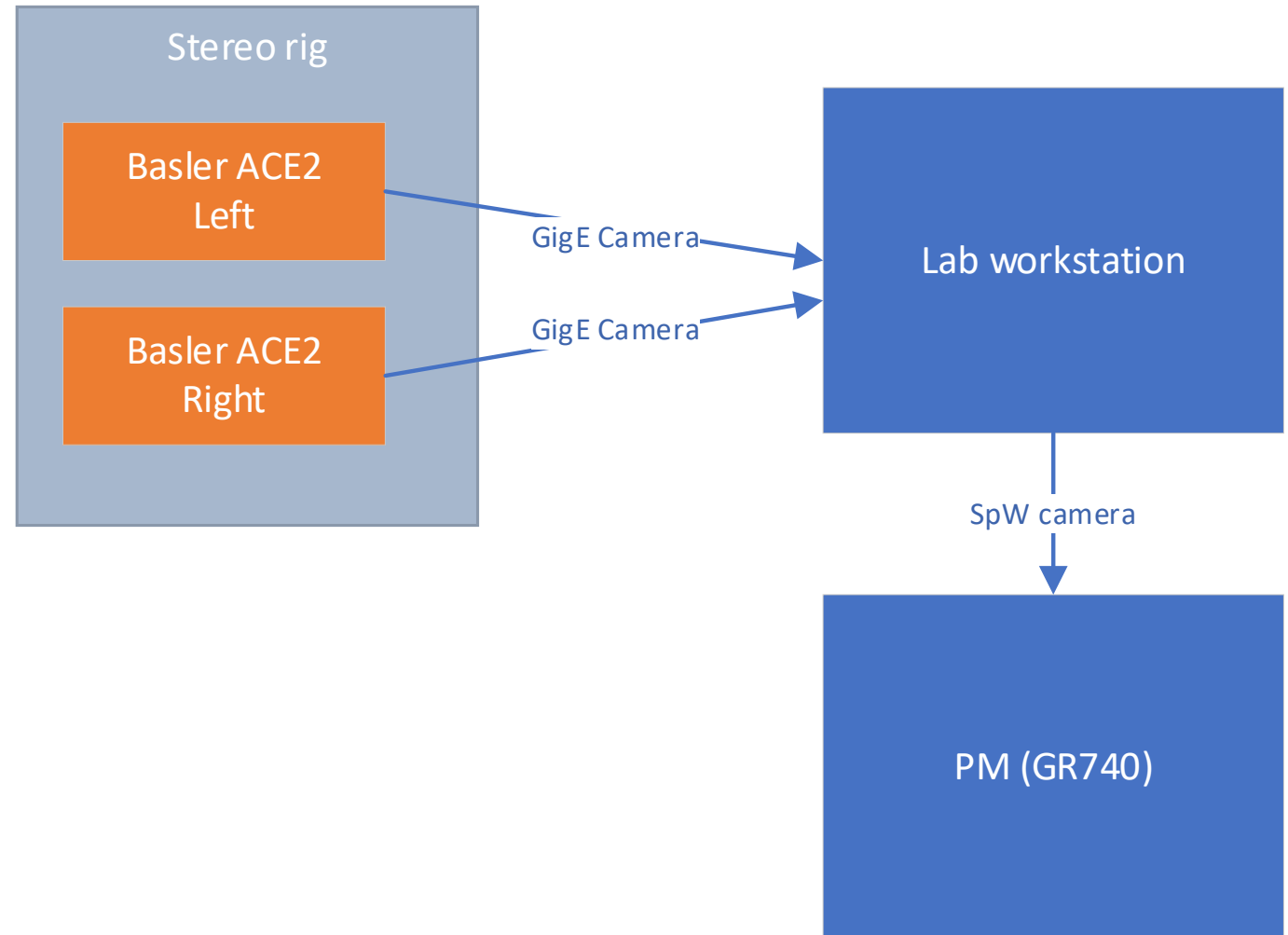
Or upload a command file:
Choose file No file chosen

Send

Sent!

RECENTLY VIEWED

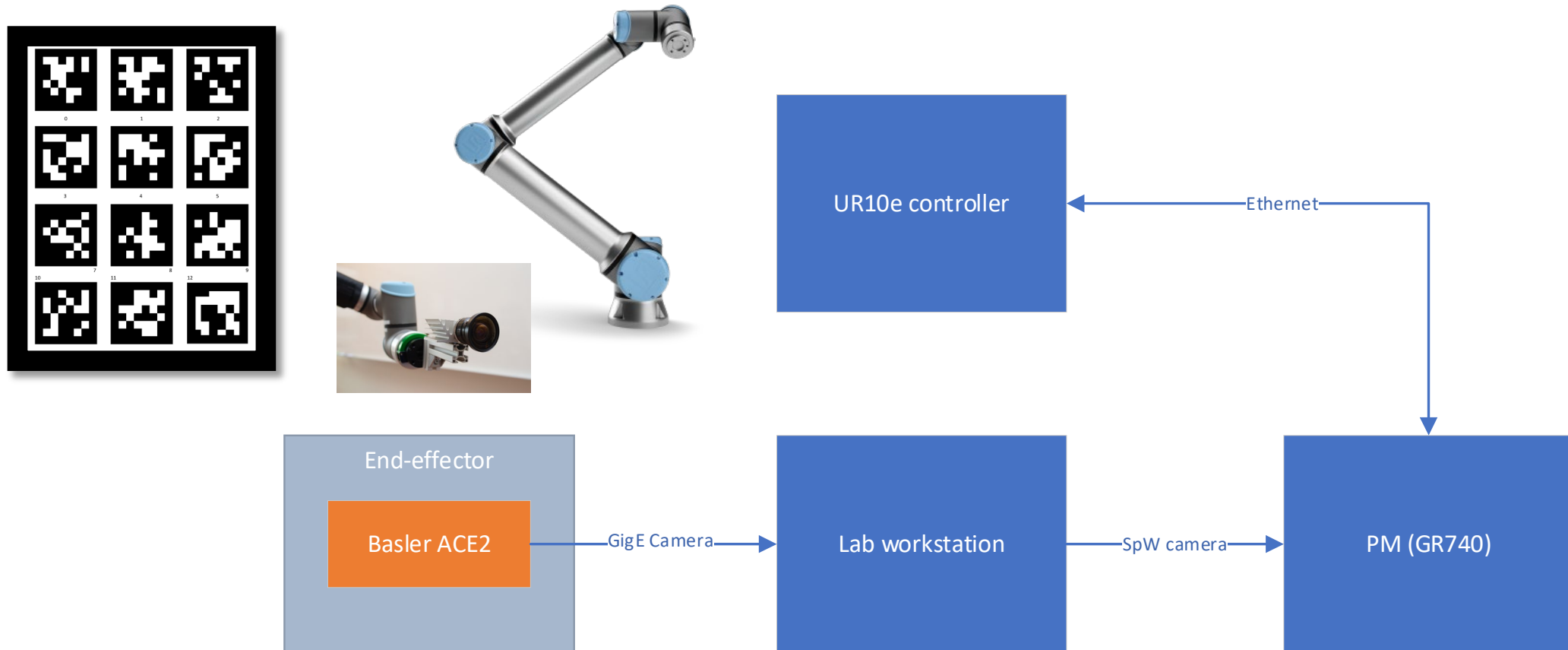
Sprint 2: Perception





SINTEF

Sprint 3: Robot arm visual servoing





SINTEF

Summary

- Future robotics applications require high-performance computer platforms for AI, image processing, machine learning etc.
- The development process from lab prototypes and into critical software according to the ECSS quality assurance standards is a hot research topic!
- Study **real-time systems** to get unique combination of cybernetical system understanding and hardware-near software development
- Join **SINTEF** and the **reliable automation** group if you want to work with practical robotics and space applications!



SINTEF

Teknologi for et bedre samfunn